Where Apertus had difficulty

TeamSwarmOps, ZHAW challenge "See It, Say It, Pick It". All measurements use `swiss-ai/Apertus-v1.5-8B`, served locally with the Swiss AI vLLM image, on the 50 public instructions (scored with ZHAW's `starter/evaluate.py`) and on 148 instructions that we wrote for the same photos. We used Apertus in four ways: alone with the plain prompt (15.1 of 65 points), alone with a two-step prompt and constrained output (40.5), as a planner over a correct scene graph (39.2) and as a reader whose output code turns into a plan (our pipeline, 65.0). The difficulties below explain most of that spread.

1. Coordinates

Apertus 1.5 accepts images, but its coordinates are not precise enough to pick an object. With the plain prompt its target pixel was on the correct object for 15 of 40 instructions. Asked to list every object with a box at 1,024 pixels, it placed 10 of 32 target boxes with an overlap of at least 0.5 with the true outline and 22 box centres on the right object, which is usable for a coarse choice but not for a grasp. With the full-resolution photo none of the 32 targets was found and coordinates scaled from 0 to 1000 found 2 of 32. We did not determine the frame that the coordinates refer to. Qwen3-VL-8B found all 32 on the same task, so in our pipeline Apertus never handles coordinates.

2. Refusal and asking

Deciding on its own, Apertus refuses too often where a plan exists and too rarely where refusal is due. With the plain prompt it asked or refused on 14 of the 40 instructions that have a plan, answered only 1 of the 10 ambiguous, impossible or unsafe instructions correctly and planned "take the mug out of my hand". Given a correct scene graph it still refused 13 of 40 valid instructions, usually with a constraint that the scene graph contradicts ("The black tray does not exist." in scene 7; "The plate cannot be picked because it is wider than the 85 mm gripper opening." for "put the mandarin on the plate", where the plate is only the destination). It also planned "pull the red gear out from under the yellow gear". Voting over six decisions did not help: 94 of our 148 instructions with voting against 104 without.

3. Spatial and physical reasoning

When Apertus writes the plan, ordering and geometry fail more often than reading. With the correct scene graph only 3 of 10 multi-step instructions were ordered correctly and "pick up the object closest to the mug" picked an object that is not the closest. A code check that rejects a pick while something still lies on the object raised the result from 90 to 104 of 148, but most remaining errors are refusals that the check cannot repair.

4. Reading instructions

As a reader that only fills in a structured query, Apertus is far more reliable: with six readings and a vote it reads all 50 public instructions correctly. The errors on our own instructions are consistent. The second object loses its colour ("closest to the yellow gear" read as closest to any gear). Photo direction and stacking are confused ("below the mug" read as stacked under it). Listed synonyms are missed ("the red block" read as an unknown object in 6 of 6 readings although the prompt says that a block is a cube). Ordinary verbs are read as unsafe ("take the cube off the gears" read as pulling it out from under them in 5 of 6 readings). The model restates "on the right" in 6 of 6 readings but fills the position field in only 2 of 6. Requiring a quote of the deciding words, checked in code against the sentence, removed one error of this kind: a choice between two fruits without a reason. Two added worked examples turned two previously correct readings among 62 instructions into errors.

5. The held-out scenes

On the five held-out scenes (40 instructions, no ground truth released) our first run gave 10 answers that we judge wrong. Eight of them were readings that contradicted the sentence in all or half of the six readings: "both" read as "any", "everything" read as "any", "below" read as "stacked under", "object" read as "part", an exception list that the sentence does not contain, a tray read as the object that something lies on and "it" not read as an unresolved pronoun. The prompt states the correct mapping for most of these, so the vote cannot repair them. A check in code that compares each reading with the words of the sentence repaired them (technical report, Sections 2.11 and 5.8). The other two errors were a blue tape at the table edge that Qwen3-VL listed as an object and a missing check that an object fits into its destination. These fixes were made after seeing the held-out scenes, so the final answers are not a clean test.

6. Effect of the larger model

Apertus 1.5 70B, with weights compressed to 8 bits on two RTX A6000, plans better than the 8B model (46.2 against 39.2 points) but reads no better. As our reader it also scored 65.0 on the public set, but its errors on our instructions were confident ones that voting does not detect: it read "cog" as a cube in 4 of 6 readings, planned the red cube for "pick up the red ball" and left "do not move the cup" out in 6 of 6 readings. Using both models, each judging the other's answer when they disagree, gave 143 of 148, the same as the 8B model alone at that point, which therefore stays our default.

7. Effective design decisions

With a JSON schema enforced by vLLM, 1 of 2,519 readings and none of 803 planner decisions failed to parse. Throwing, pushing off, pulling out from under and taking from a hand were recognised in every public instruction. Six readings take about 12 seconds per instruction on one RTX A6000. Apertus does reliably what a reader does, namely filling fixed fields with constrained output. Code does what Apertus does unreliably: coordinates, distances, stacking order and the physical checks. When three of six readings say ask or refuse, the pipeline does, because a missed refusal costs more than an unnecessary question. This division accounts for the step from 39.2 to 65.0 points.